МИНОБРНАУКИ РОССИИ

Федеральное государственное автономное образовательное учреждения высшего образования

«ЮЖНЫЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»

Институт компьютерных технологий и информационной безопасности

Кафедра математического обеспечения и применения ЭВМ

ЛАБОРАТОРНАЯ РАБОТА № 1

по дисциплине

«Программирование компьютерной графики»

на тему:

«Средства вывода графики»

Бригада №24

Вариант №3

Выполнил(и):

Студент(ы)группы

КТбо4-6

_____ Таржуткин Е.М.

подпись

Проверил:

Ассистент кафедры

МОП ЭВМ

_____ Гуляев Н. А.

подпись

Оценка

_____

«22» сентября 2026 г.

Таганрог 2026

СОДЕРЖАНИЕ

Содержание 2
1. Цель работы 3
2. Задание 3
3. Описание использованных алгоритмов 4
4. Описание использованных средств 6
5. Описание функционала разработанного программного приложения 6
6. Описание полученных результатов 7
7. Заключение 8
Приложение (графический материал) 9
Приложение (листинг) 18

1. Цель работы

Практическая цель: разработка программного модуля, реализующего отображение динамического (изменяющегося) растрового изображения на устройстве вывода.

Задачи:

1. Изучение математического аппарата компьютерной графики.
2. Разработка алгоритмов на базе используемого математического аппарата (движение, обработка точек и фигур).
3. Проектирование, разработка и отладка программного модуля.
4. Проверка работоспособности на разных наборах исходных данных.
5. Подготовка документации и компоновка материалов.

2. Задание

Вариант выполнения: ИВ3 — индивидуальное выполнение.

Среда разработки и платформа:

- Язык программирования: Turbo Assembler (TASM).
- ОС: DOS 6.xx (в том числе DosBox).
- Вывод графики: прерывания int 10h, графический режим VGA 13h (320×200, 256 цветов).

Обязательные рабочие задачи:

- ОЗ.Р1 — разработка программной логики реализации смены положения движущихся фигур;
- ОЗ.Р2 — разработка программной логики вывода изображения (кадра);
- ОЗ.Д1 — оформление и представление результатов.

Дополнительные рабочие задачи:

- ДЗ.А1 — использование равностороннего треугольника в качестве типа габаритов фигуры (без вращения), разработка алгоритма и программной реализации определения выхода за границы области движения;
- ДЗ.Б1 — дополнение движения фигуры постепенным вращением относительно точки центра;
- ДЗ.В5 — замена равномерного прямолинейного движения фигуры движением с заданной изначальной скоростью с учётом силы тяготения, при столкновении с границами — «отскок» по закону отражения луча с приданием импульса;
- ДЗ.Е3 — оптимизация отрисовки за счёт схемы «двойной буферизации», с возможностью принудительной установки FPS и размера области вывода.

Распределение задач: все задачи выполнены единственным исполнителем.

3. Описание использованных алгоритмов

3.1 Математическая модель движения (ДЗ.В5)

- Движение фигуры описывается уравнениями баллистики:
- $vx=v0x$;
- $vy=v0y+g\cdot t$;
- $x=x0+v0x\cdot t$;
- $y=y0+v0y\cdot t+2g\cdot t2$.
- При столкновении с границей области движения выполняется отскок по закону отражения:
- при ударе о вертикальную границу: $vx=-vx$;
- при ударе о горизонтальную границу: $vy=-vy$, затем $vy=vy-$impulse, причем добавление импульса лишь при ударе о нижнюю границу

3.2 Поворот фигуры (ДЗ.Б1)

- Поворот вершин равностороннего треугольника относительно центра на угол φ:
- $x' = x0 + r \cdot \cos\varphi$;
- $y' = y0 + r \cdot \sin\varphi$;
- Для вычисления синусов и косинусов используется таблица sin_tab (256 слов), что обеспечивает целочисленную арифметику с фиксированной точкой (точность 1/256).

3.3 Определение выхода за границы (ДЗ.А1)

- Габариты фигуры описываются равносторонним треугольником. Для проверки выхода за границы области перемещения вычисляются минимальные и максимальные координаты вершин:
- $\text{minX} = \min(xi)$, $\text{maxX} = \max(xi)$;
- $\text{minY} = \min(yi)$, $\text{maxY} = \max(yi)$.
- Если $\text{minX} < 0$, $\text{maxX} > \text{viewW}$, $\text{minY} < 0$ или $\text{maxY} > \text{viewH}$ — фигура вышла за границы, выполняется коррекция положения и отскок.

3.4 Двойная буферизация (ДЗ.Е3)

- Отрисовка ведётся в «задний буфер» (страница видеопамяти VGA), после чего содержимое буфера копируется на экран. Это устраняет мерцание и позволяет независимо управлять FPS и размером области вывода.

4. Описание использованных средств

- **Язык программирования:** Turbo Assembler (TASM).
- **Платформа:** DOS 6.xx (DosBox).
- **Графический режим:** VGA 13h (320×200, 256 цветов).
- **Вывод графики:** прерывания int 10h (установка режима), прямая запись в видеопамять A000h.
- **Ввод:** прерывания int 16h (клавиатура).
- **Таймер:** прерывание int 15h, функция 86h (задержка).

  Сторонние библиотеки не использовались.

5. Описание функционала разработанного программного приложения

5.1 Структура программы

- main — точка входа, главный цикл.
- gfx_init / gfx_shutdown — инициализация и завершение видеорежима.
- handle_input — обработка клавиатуры (выход, изменение FPS, размера области).
- color_view — перерисовка области вывода при изменении размера.
- fill_rect — заливка прямоугольной области.
- draw_triangle — отрисовка треугольника.
- draw_line — отрисовка линии (алгоритм Брезенхэма).
- update_body — обновление физики (движение, поворот, вычисление вершин).
- resolve_collision — обработка столкновений с границами.

5.2 Структуры данных

- Triangle — угол, приращение угла, сторона, координаты вершин.
- Body — треугольник, позиция, скорость, гравитация, импульс.
- Point — координаты x, y.

5.3 Основной рабочий цикл

1. Обработка ввода.
2. Стирание предыдущего положения фигуры.
3. Обновление физики (движение, поворот, вычисление вершин).
4. Отрисовка нового положения.
5. Задержка для поддержания заданного FPS.

5.4 Взаимодействие с пользователем

- Esc — выход.
- + / = — увеличение FPS.
- - / _ — уменьшение FPS.
- [— уменьшение области вывода.
-] — увеличение области вывода.

6. Описание полученных результатов

В ходе выполнения работы разработан программный модуль на TASM, реализующий:

- динамическое отображение равностороннего треугольника;
- баллистическое движение с гравитацией и отскоком (ДЗ.В5);
- постепенное вращение фигуры (ДЗ.Б1);
- определение выхода за границы (ДЗ.А1);
- двойную буферизацию и управление FPS (ДЗ.Е3).

Результаты исследований (ДЗ.Е3):

FPS	Размер области	Артефакты (исходный)	Артефакты (оптимизированный)
30	320×200	мерцание	отсутствуют
45	320×200	мерцание	отсутствуют
60	320×200	мерцание	отсутствуют
120	320×200	мерцание	отсутствуют
60	160×100	неполная прорисовка	отсутствуют

Вывод: применение двойной буферизации позволило устранить мерцание и неполную прорисовку, а также обеспечить независимое управление FPS и размером области вывода.

7. Заключение

В ходе лабораторной работы были освоены навыки разработки интерактивных графических приложений на языке ассемблера для платформы DOS. Реализованы все обязательные и дополнительные задачи варианта ИВ3. Программный модуль успешно функционирует, демонстрирует динамическое изменение графического содержимого и корректную обработку столкновений. Применение двойной буферизации позволило повысить качество отрисовки и устранить артефакты.

ПРИЛОЖЕНИЕ (ГРАФИЧЕСКИЙ МАТЕРИАЛ)

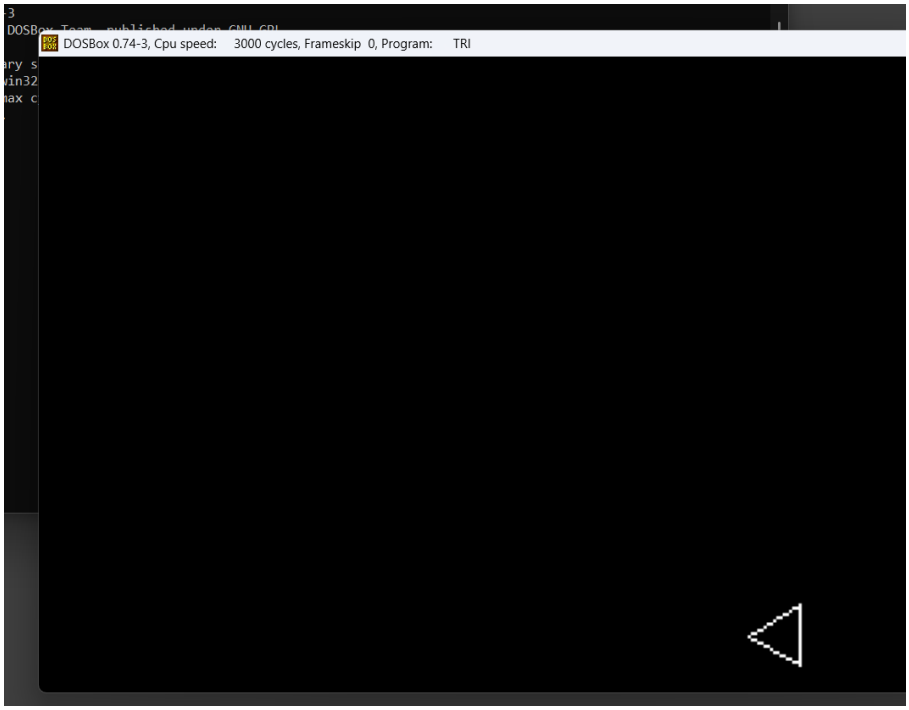


Рисунок 1 -Треугольник прыгает по всему окну

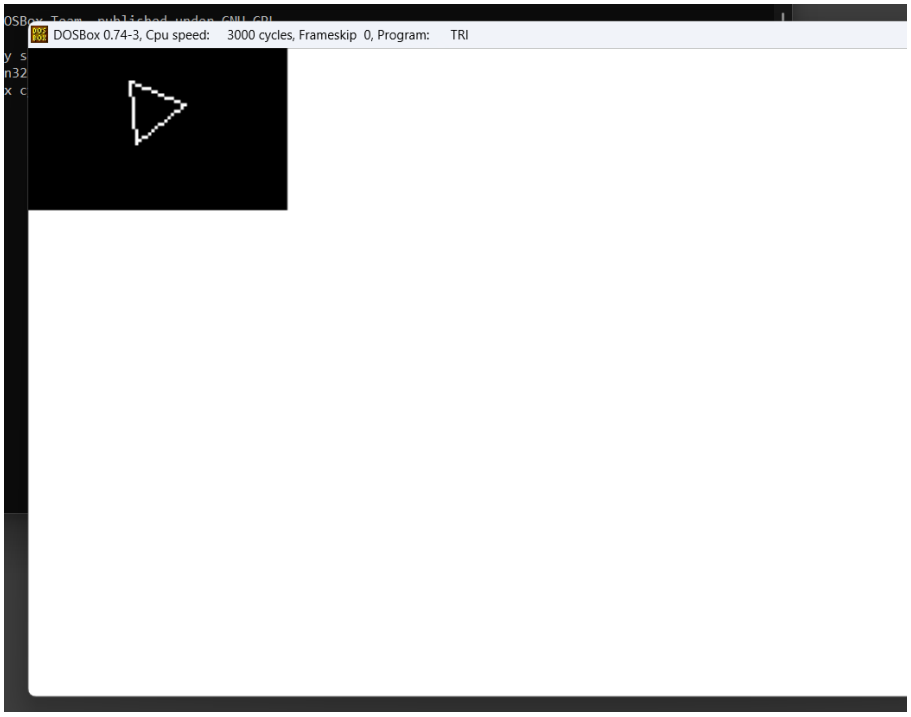


Рисунок 2 – С помощью клавиш уменьшили края, где может находится треугольник

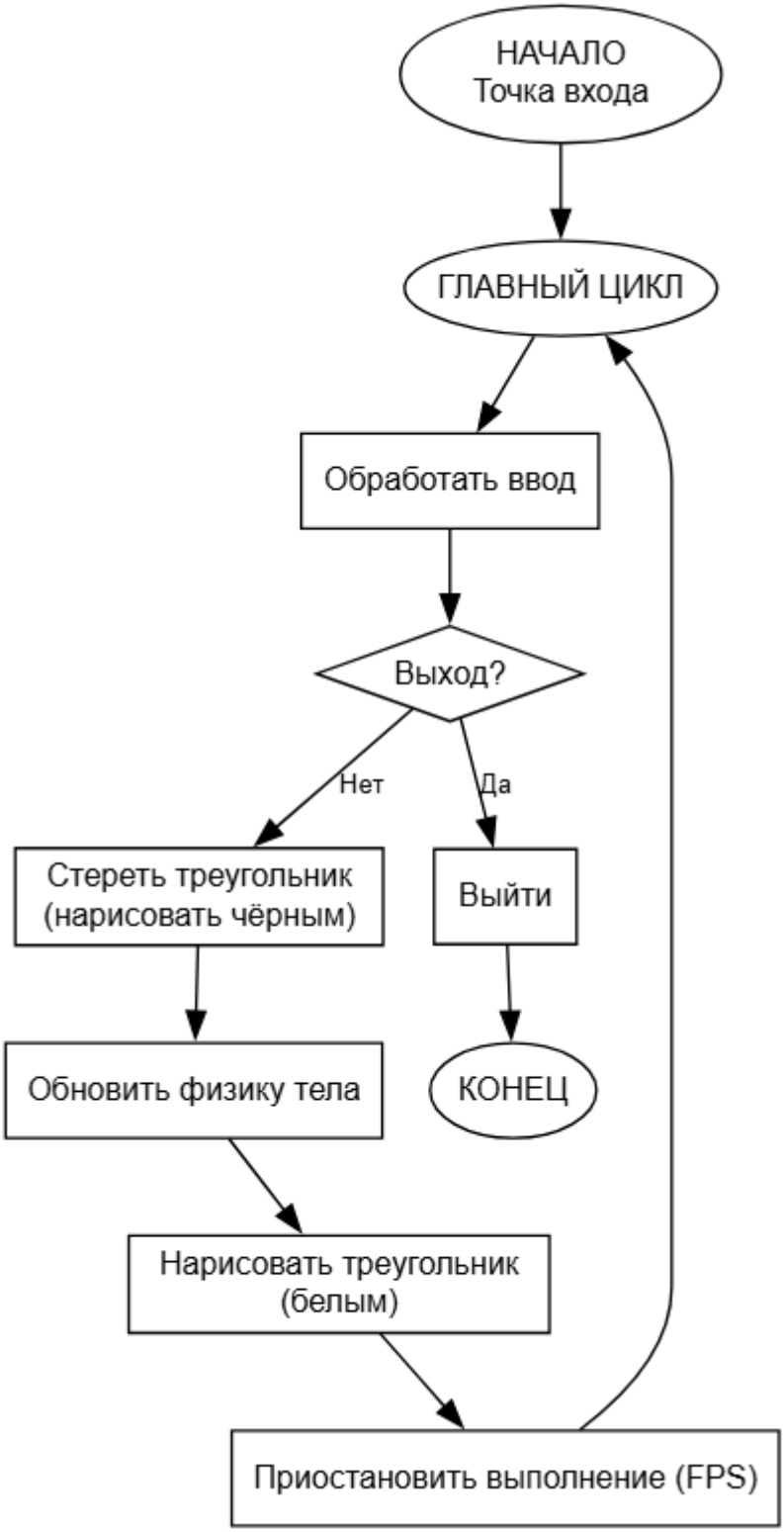


Рисунок 3 Главная функция

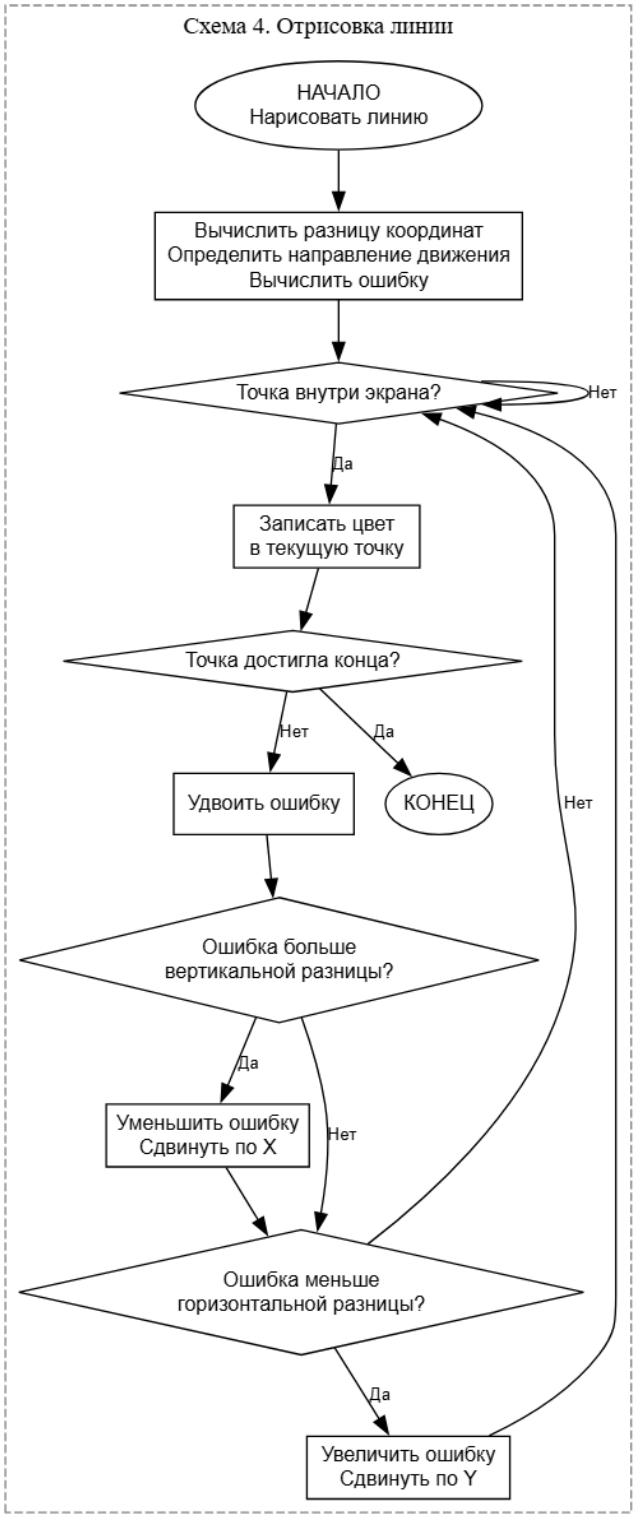


Рисунок 4 Функция отрисовки линии



Рисунок 5 Заполнение прямоугольника цветом



Рисунок 6 Очистка экрана

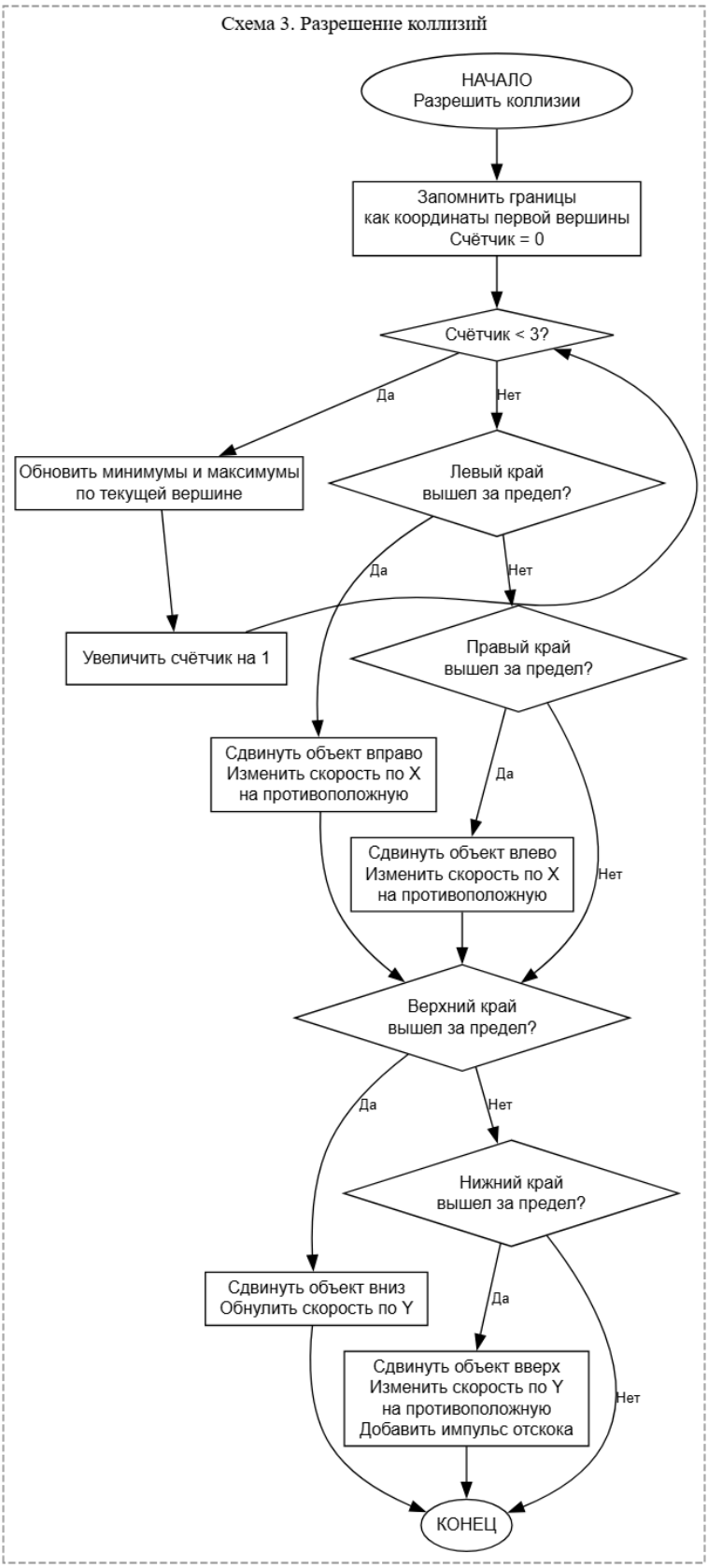


Рисунок 7 Разрешение колизий

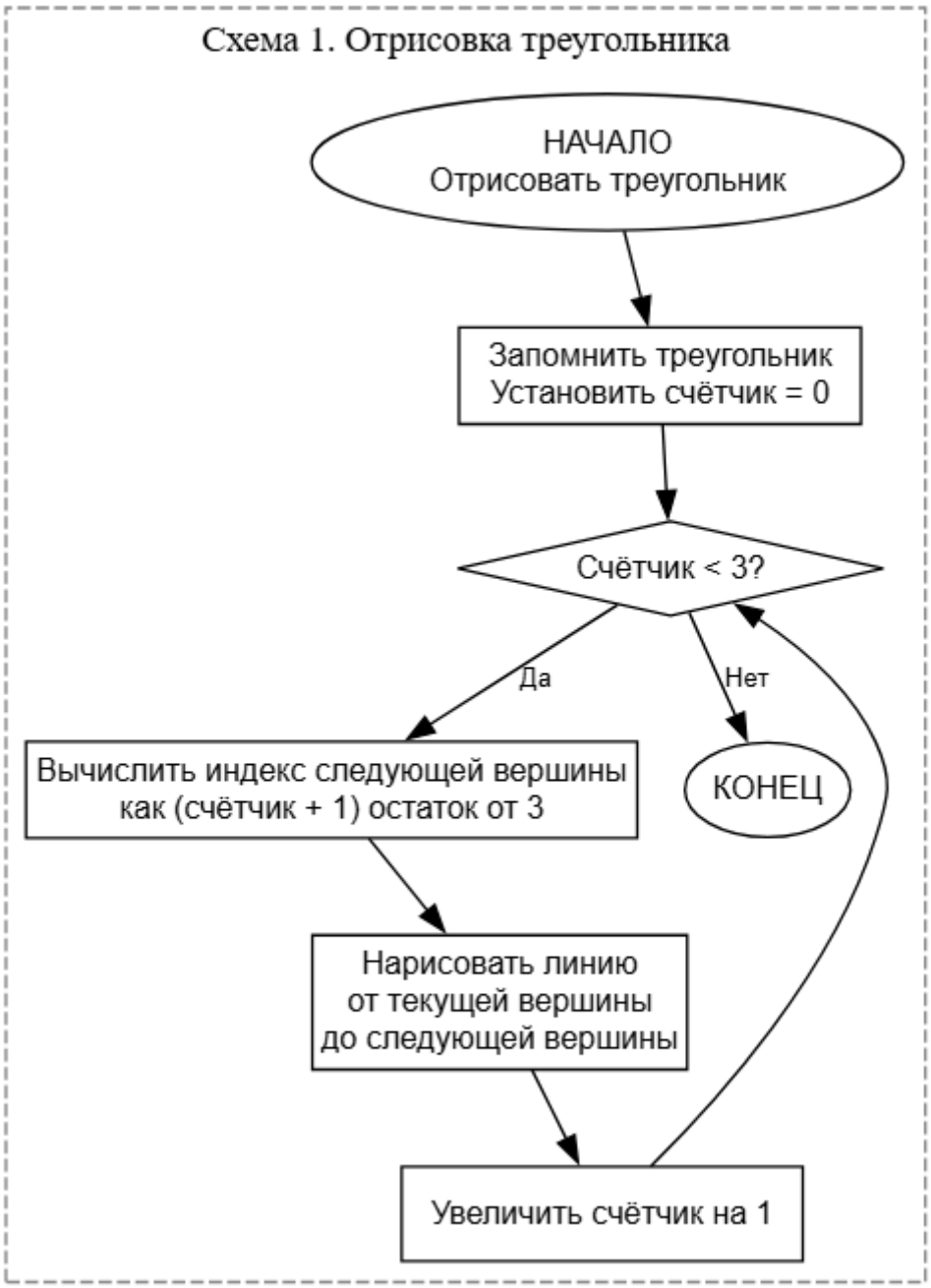


Рисунок 8 Функция рисования треугольника



Рисунок 9 Функция вычисления вершин треугольника по углу

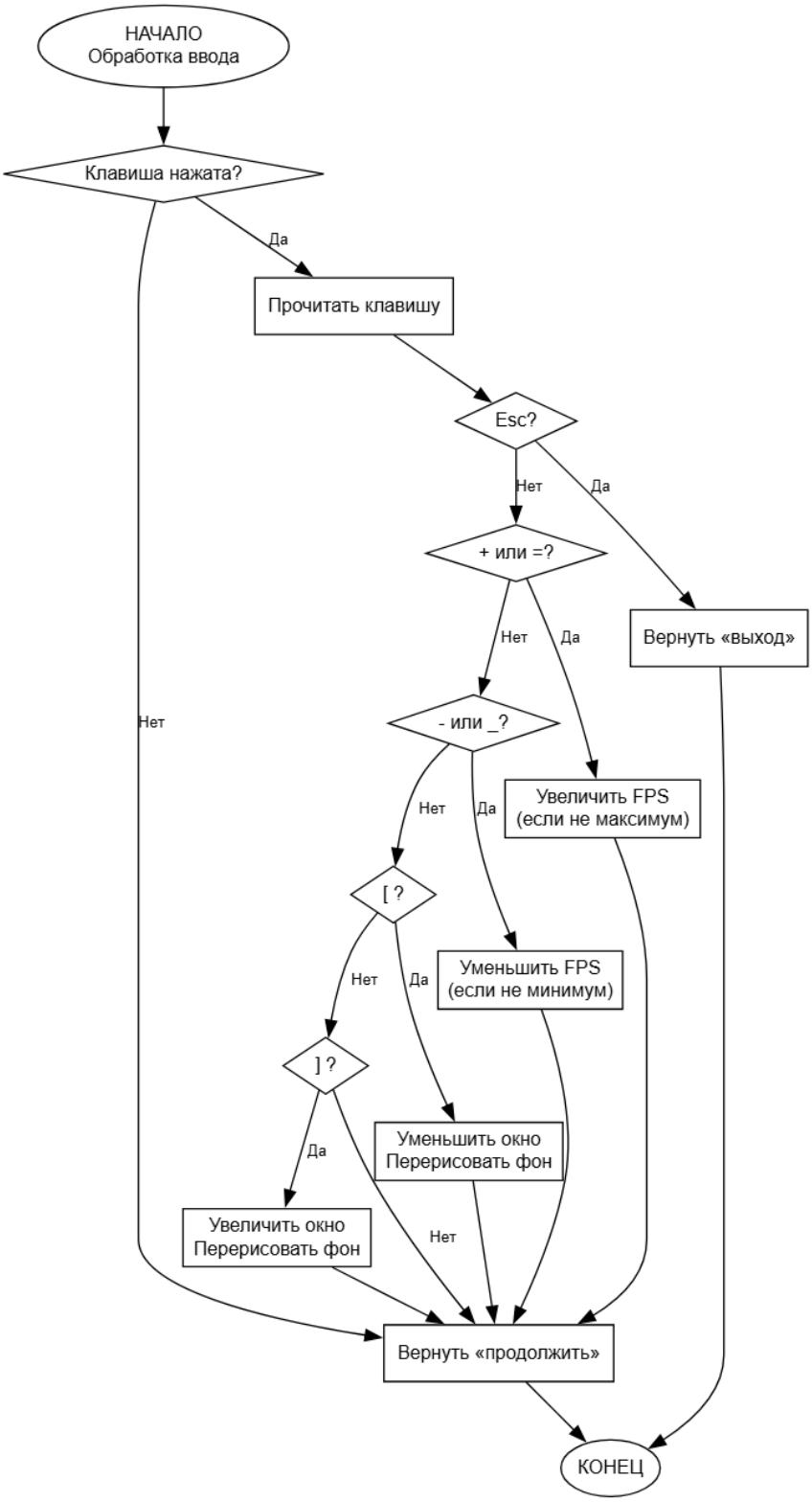


Рисунок 10 Обработка ввода

ПРИЛОЖЕНИЕ (ЛИСТИНГ)

Tri.asm

```
.model small
.stack 100h

; ============================================================
; КОНСТАНТЫ
; ============================================================

;  Экран
SCREEN_W    equ 320
SCREEN_H    equ 200

;  Цвета
BLACK       equ 0
WHITE       equ 15

;  Смещения Triangle
TRI_ANGLE   equ 0
TRI_DANGLE  equ 2
TRI_SIDE    equ 4
TRI_POINTS  equ 6
TRI_SIZE    equ 18

;  Смещения Body
BODY_TRI     equ 0
BODY_POS     equ 18
BODY_VEL     equ 22
BODY_G       equ 26
BODY_IMPULSE equ 28
BODY_SIZE    equ 30

;  Смещения Point
PT_X        equ 0
PT_Y        equ 2
PT_SIZE     equ 4

; ============================================================
; СЕГМЕНТ ДАННЫХ (инициализированные данные)
; ============================================================

_DATA SEGMENT
```

```
;  Простые переменные
viewW      dw 320
prev_view_w dw 320
viewH      dw 200
prev_view_h dw 200
fps        dw 60

;  Видеопамять (far-указатель)
vram_off    dw 0
vram_seg    dw 0A000h

;  Body (сразу с начальными значениями)
body    LABEL WORD;CS unreachable from current segment
; Triangle (18 байт):
dw      0             ; tri.angle = 0
dw      10            ; tri.dangle = 10
dw      20            ; tri.side = 20
dw      0, 0          ; points[0].x, .y
dw      0, 0          ; points[1].x, .y
dw      0, 0          ; points[2].x, .y
; Point pos (4 байта):
dw      160           ; pos.x = 160
dw      100           ; pos.y = 100
; Point vel (4 байта):
dw      2             ; vel.x = 2
dw      0            ; vel.y = -3
; g, impulse:
dw      1             ; g = 1
dw      1             ; impulse = 1

;  Таблица синусов (256 слов)
sin_tab LABEL WORD
dw      0, 6, 13, 19, 25, 31, 38, 44
dw      50, 56, 62, 68, 74, 80, 86, 92
dw      98, 104, 109, 115, 121, 126, 132, 137
dw      142, 147, 152, 157, 162, 167, 171, 176
dw      181, 185, 189, 193, 197, 201, 205, 209
dw      212, 216, 219, 222, 225, 228, 231, 234
dw      236, 239, 241, 243, 245, 247, 248, 250
dw      251, 252, 253, 254, 254, 255, 255, 255
dw      256, 255, 255, 255, 254, 254, 253, 252
dw      251, 250, 248, 247, 245, 243, 241, 239
```

```
dw      236, 234, 231, 228, 225, 222, 219, 216
dw      212, 209, 205, 201, 197, 193, 189, 185
dw      181, 176, 171, 167, 162, 157, 152, 147
dw      142, 137, 132, 126, 121, 115, 109, 104
dw      98, 92, 86, 80, 74, 68, 62, 56
dw      50, 44, 38, 31, 25, 19, 13, 6
dw      0, -6, -13, -19, -25, -31, -38, -44
dw      -50, -56, -62, -68, -74, -80, -86, -92
dw      -98, -104, -109, -115, -121, -126, -132, -137
dw      -142, -147, -152, -157, -162, -167, -171, -176
dw      -181, -185, -189, -193, -197, -201, -205, -209
dw      -212, -216, -219, -222, -225, -228, -231, -234
dw      -236, -239, -241, -243, -245, -247, -248, -250
dw      -251, -252, -253, -254, -254, -255, -255, -255
dw      -256, -255, -255, -255, -254, -254, -253, -252
dw      -251, -250, -248, -247, -245, -243, -241, -239
dw      -236, -234, -231, -228, -225, -222, -219, -216
dw      -212, -209, -205, -201, -197, -193, -189, -185
dw      -181, -176, -171, -167, -162, -157, -152, -147
dw      -142, -137, -132, -126, -121, -115, -109, -104
dw      -98, -92, -86, -80, -74, -68, -62, -56
dw      -50, -44, -38, -31, -25, -19, -13, -6


;  Переменные, которые не нужно инициализировать
r_factor    dw ?        ; радиус описанной окружности (fixed-point)
angle_idx   dw ?        ; текущий угол (0..255)
local_idx   dw ?        ; индекс в sin_tab для текущей вершины
; для resolve_collision
minX        dw ?
minY        dw ?
maxX        dw ?
maxY        dw ?
; для функций разных
x    dw ?
y    dw ?
x0   dw ?
x1   dw ?
y0   dw ?
y1   dw ?
col  dw ?
; для draw_line
dl_dx   dw ?
```

```
 dl_dy   dw ?
 dl_sx   dw ?
 dl_sy   dw ?
 dl_err  dw ?
 ; для color view
 cv_tl  dw ?, ?     ; x, y
 cv_br  dw ?, ?     ; x, y

_DATA ENDS


; ============================================================
; СЕГМЕНТ КОДА
; ============================================================

_TEXT SEGMENT
 ASSUME CS:_TEXT, DS:_DATA


 ;  Точка входа
 main proc near
  mov  ax, _DATA
  mov  ds, ax
  ;  Инициализация видеорежима
  call gfx_init

;  Главный цикл
main_loop:
  call handle_input
  test ax, ax         ; если 0 — выход
  jz   main_exit
  mov  al, BLACK
  call draw_triangle  ; стираем
  call update_body    ; обновляем физику
  mov  al, WHITE
  call draw_triangle  ; рисуем

  ;  Задержка
  mov  ax, 1000
  xor  dx, dx
  div  word ptr [fps]     ; ax = мс
  mov  cx, 1000
  mul  cx                 ; dx:ax = мкс
  mov  cx, dx
```

```
  mov  dx, ax
  mov  ah, 86h
  int  15h

  jmp  main_loop

main_exit:
  call gfx_shutdown

  ;  Выход в DOS
  mov  ax, 4C00h
  int  21h

 main endp

 ; ============================================================
 ; void gfx_init(void)
 ; Устанавливает VGA режим 13h (320x200x256)
 ; Инициализирует vram_off, vram_seg
 ; ============================================================
 gfx_init proc near
  ;  Установить видеорежим 13h
  mov ax, 0013h       ; ah = 00h (set video mode), al = 13h (320x200x256)
  int 10h             ; вызов BIOS
  ;  Инициализировать vram
  mov word ptr [vram_off], 0      ; offset = 0
  mov word ptr [vram_seg], 0A000h ; segment = A000h
  ret
 gfx_init endp

 ; ============================================================
 ; void gfx_shutdown(void)
 ; Возвращает текстовый режим 03h
 ; ============================================================
 gfx_shutdown proc near
  ;  Вернуть текстовый режим
  mov ax, 0003h       ; ah = 00h (set video mode), al = 03h (text mode)
  int 10h             ; вызов BIOS
  ret
 gfx_shutdown endp


 ; ============================================================
```

```
 ; handle_input
 ;   выход: ax = 1 — продолжить, 0 — выход
 ; ============================================================
 handle_input proc near
  ; --- kbhit? ---
  mov  ah, 01h
  int  16h
  jz   hi_cont              ; ZF=1 → нет нажатия

  ; --- getch ---
  xor  ah, ah
  int  16h                  ; al = ASCII, ah = scan

  cmp  al, 27
  je   hi_exit

  cmp  al, '+'
  je   hi_fps_up
  cmp  al, '='
  je   hi_fps_up

  cmp  al, '-'
  je   hi_fps_dn
  cmp  al, '_'
  je   hi_fps_dn

  cmp  al, '['
  je   hi_shrink

  cmp  al, ']'
  je   hi_grow

  jmp  hi_cont

; ------------------------------------------------------------
hi_fps_up:
  cmp  word ptr [fps], 300
  jge  hi_cont
  add  word ptr [fps], 60
  jmp  hi_cont

; ------------------------------------------------------------
hi_fps_dn:
```

```
  cmp  word ptr [fps], 30
  jle  hi_cont
  sub  word ptr [fps], 30
  jmp  hi_cont

; ------------------------------------------------------
hi_shrink:
  cmp  word ptr [viewW], 80
  jle  hi_sh_h
  sub  word ptr [viewW], 40
hi_sh_h:
  cmp  word ptr [viewH], 50
  jle  hi_sh_done
  sub  word ptr [viewH], 25
hi_sh_done:
  call color_view
  jmp  hi_cont

; ------------------------------------------------------
hi_grow:
  cmp  word ptr [viewW], SCREEN_W
  jge  hi_gr_h
  add  word ptr [viewW], 40
hi_gr_h:
  cmp  word ptr [viewH], SCREEN_H
  jge  hi_gr_done
  add  word ptr [viewH], 25
hi_gr_done:
  call color_view

; ------------------------------------------------------
hi_cont:
  mov  ax, 1
  ret

hi_exit:
  xor  ax, ax
  ret
 handle_input endp



 ; ======================================================
```

```
 ; color_view
 ;   вход:  -
 ;   портит: ax, bx, cx, dx, si, di, bp
 ; ============================================================
 color_view proc near
  ; --- по горизонтали ---
  mov  ax, [viewW]
  cmp  ax, [prev_view_w]
  je   cv_vert

  mov  si, offset cv_tl
  mov  di, offset cv_br

  jg   cv_wider

  ; w < pw : fill_rect(w, 0, pw, ph, WHITE)
  mov  [si + PT_X], ax
  mov  word ptr [si + PT_Y], 0
  mov  bx, [prev_view_w]
  mov  [di + PT_X], bx
  mov  bx, [prev_view_h]
  mov  [di + PT_Y], bx
  mov  ax, WHITE
  mov [col], ax
  call fill_rect
  jmp  cv_vert

cv_wider:
  ; w > pw : fill_rect(pw, 0, w, h, BLACK)
  mov  bx, [prev_view_w]
  mov  [si + PT_X], bx
  mov  word ptr [si + PT_Y], 0
  mov  [di + PT_X], ax
  mov  bx, [viewH]
  mov  [di + PT_Y], bx
  mov  ax, BLACK
  mov [col], ax
  call fill_rect

cv_vert:
  ; --- по вертикали ---
  mov  ax, [viewH]
  cmp  ax, [prev_view_h]
```

```
  je   cv_done

  mov  si, offset cv_tl
  mov  di, offset cv_br

  jg   cv_taller

  ; h < ph : fill_rect(0, h, pw, ph, WHITE)
  mov  word ptr [si + PT_X], 0
  mov  [si + PT_Y], ax
  mov  bx, [prev_view_w]
  mov  [di + PT_X], bx
  mov  bx, [prev_view_h]
  mov  [di + PT_Y], bx
  mov  ax, WHITE
  mov [col], ax
  call fill_rect
  jmp  cv_done

cv_taller:
  ; h > ph : fill_rect(0, ph, w, h, BLACK)
  mov  word ptr [si + PT_X], 0
  mov  bx, [prev_view_h]
  mov  [si + PT_Y], bx
  mov  bx, [viewW]
  mov  [di + PT_X], bx
  mov  [di + PT_Y], ax
  mov  ax, BLACK
  mov [col], ax
  call fill_rect

cv_done:
  mov  ax, [viewW]
  mov  [prev_view_w], ax
  mov  ax, [viewH]
  mov  [prev_view_h], ax
  ret
 color_view endp
```

```
; ============================================================
; fill_rect
;   вход:  si -> точка A (x0, y0)
;          di -> точка B (x1, y1)
;          col = цвет
;   портит: ax
; ============================================================
fill_rect proc near
 push bx
 push cx
 push dx
 push bp
 push si
 push di

 ; --- сохраняем координаты в BSS ---
 mov  ax, [si + PT_X]
 mov  [x0], ax
 mov  ax, [si + PT_Y]
 mov  [y0], ax
 mov  ax, [di + PT_X]
 mov  [x1], ax
 mov  ax, [di + PT_Y]
 mov  [y1], ax

 ; ============================================================
 ; clip: x0 = max(x0, 0), y0 = max(y0, 0)
 ; ============================================================
 cmp  word ptr [x0], 0
 jge  fr_x0_ok
 mov  word ptr [x0], 0
fr_x0_ok:
 cmp  word ptr [y0], 0
 jge  fr_y0_ok
 mov  word ptr [y0], 0
fr_y0_ok:

 ; ============================================================
```

```
  ; clip: x1 = min(x1, viewW), y1 = min(y1, viewH)
  ; ======================================================
  mov  ax, SCREEN_W
  cmp  [x1], ax
  jle  fr_x1_ok
  mov  [x1], ax
fr_x1_ok:
  mov  ax, SCREEN_H
  cmp  [y1], ax
  jle  fr_y1_ok
  mov  [y1], ax
fr_y1_ok:

  ; ======================================================
  ; если x0 >= x1 или y0 >= y1 — нечего рисовать
  ; ======================================================
  mov  ax, [x0]
  cmp  ax, [x1]
  jge  fr_end
  mov  ax, [y0]
  cmp  ax, [y1]
  jge  fr_end

  ; ======================================================
  ; устанавливаем es = vram_seg (один раз)
  ; ======================================================
  mov  ax, [vram_seg]
  mov  es, ax

  ; ======================================================
  ; внешний цикл: y от y0 до y1-1
  ; ======================================================
  mov  ax, [y0]
  mov  [y], ax

fr_y_loop:
  mov  ax, [y]
  cmp  ax, [y1]
  jge  fr_end

  ; --- bp = y * SCREEN_W + x0 ---
  mov  ax, [y]
  mov  bx, SCREEN_W
```

```
  imul bx                 ; dx:ax = y * SCREEN_W
  add  ax, [x0]           ; ax = y*SCREEN_W + x0
  mov  bp, ax

  ; --- di = vram_off + bp ---
  mov  di, [vram_off]
  add  di, bp

  ; --- ax = цвет, cx = количество пикселей ---
  mov  al, byte ptr [col]
  mov  ah, al             ; ah = al, чтобы писать слово (2 байта) за раз
  mov  cx, [x1]
  sub  cx, [x0]           ; cx = ширина

  ; ========================================================
  ; внутренний цикл: пишем cx байт
  ; ========================================================
  ; пишем словами (по 2 байта) — быстрее, чем побайтно
  mov  bx, cx
  shr  bx, 1              ; bx = cx / 2 (число слов)
  jz   fr_bytes           ; если < 2 байт — только байты

fr_word_loop:
  mov  es:[di], ax        ; пишем слово (al + ah = цвет + цвет)
  add  di, 2
  dec  bx
  jnz  fr_word_loop

fr_bytes:
  test cx, 1              ; остался ли нечётный байт?
  jz   fr_next_row
  mov  es:[di], al        ; пишем один байт

fr_next_row:
  inc  word ptr [y]
  jmp  fr_y_loop

fr_end:
  pop  di
  pop  si
  pop  bp
  pop  dx
  pop  cx
```

```
 pop  bx
 ret
fill_rect endp



; ============================================================
; draw_triangle
; ============================================================
draw_triangle proc near
 push bx
 push si
 push di

 mov  byte ptr [col], al
 mov  bx, offset body
 add  bx, BODY_TRI + TRI_POINTS    ; bx -> points[0]

 ; --- сторона 0-1 ---
 mov  si, bx
 lea  di, [bx + 1*PT_SIZE]
 call draw_line

 ; --- сторона 1-2 ---
 lea  si, [bx + 1*PT_SIZE]
 lea  di, [bx + 2*PT_SIZE]
 call draw_line

 ; --- сторона 2-0 ---
 lea  si, [bx + 2*PT_SIZE]
 mov  di, bx
 call draw_line

 pop  di
 pop  si
 pop  bx
 ret
draw_triangle endp


; mov  bx, offset body
; ; сторона 0-1
; lea  si, [bx + BODY_TRI + TRI_POINTS + 0*PT_SIZE]
; lea  di, [bx + BODY_TRI + TRI_POINTS + 1*PT_SIZE]
```

```
 ; mov  al, WHITE
 ; call draw_line
 ; ; сторона 1-2
 ; lea  si, [bx + BODY_TRI + TRI_POINTS + 1*PT_SIZE]
 ; lea  di, [bx + BODY_TRI + TRI_POINTS + 2*PT_SIZE]
 ; mov  al, WHITE
 ; call draw_line
 ; ; сторона 2-0
 ; lea  si, [bx + BODY_TRI + TRI_POINTS + 2*PT_SIZE]
 ; lea  di, [bx + BODY_TRI + TRI_POINTS + 0*PT_SIZE]
 ; mov  al, WHITE
 ; call draw_line
 ; ============================================================
 ; draw_line
 ;   вход:  si -> точка A (PT_X, PT_Y)
 ;          di -> точка B
 ;          col = цвет
 ;   портит: ax
 ; ============================================================
 draw_line proc near
  push bx
  push cx
  push dx
  push bp
  push si
  push di

  mov  ax, [si + PT_X]
  mov  [x0], ax
  mov  ax, [si + PT_Y]
  mov  [y0], ax
  mov  ax, [di + PT_X]
  mov  [x1], ax
  mov  ax, [di + PT_Y]
  mov  [y1], ax

  ; dx = |x1 - x0|
  mov  ax, [x1]
  sub  ax, [x0]
  jns  dl_dx_pos
  neg  ax
dl_dx_pos:
  mov  [dl_dx], ax
```

```
  ; dy = |y1 - y0|
  mov  ax, [y1]
  sub  ax, [y0]
  jns  dl_dy_pos
  neg  ax
dl_dy_pos:
  mov  [dl_dy], ax

  ; sx
  mov  ax, [x0]
  cmp  ax, [x1]
  jl   dl_sx_pos
  mov  word ptr [dl_sx], -1
  jmp  dl_sx_done
dl_sx_pos:
  mov  word ptr [dl_sx], 1
dl_sx_done:

  ; sy
  mov  ax, [y0]
  cmp  ax, [y1]
  jl   dl_sy_pos
  mov  word ptr [dl_sy], -1
  jmp  dl_sy_done
dl_sy_pos:
  mov  word ptr [dl_sy], 1
dl_sy_done:

  ; err = dx - dy
  mov  ax, [dl_dx]
  sub  ax, [dl_dy]
  mov  [dl_err], ax

dl_loop:
  ; --- clip ---
  mov  ax, [x0]
  or   ax, ax
  js   dl_skip_put
  cmp  ax, [viewW]
  jge  dl_skip_put

  mov  bx, [y0]
```

```
  or   bx, bx
  js   dl_skip_put
  cmp  bx, [viewH]
  jge  dl_skip_put

  ; --- bp = y0 * SCREEN_W + x0 ---
  mov  ax, [y0]            ; ax = y0
  mov  bx, SCREEN_W        ; bx = SCREEN_W (константа)
  imul bx                  ; dx:ax = y0 * SCREEN_W
  ; для y0 < 2048 результат влезает в ax
  add  ax, [x0]            ; ax = y0*SCREEN_W + x0
  mov  bp, ax              ; bp = смещение

  ; --- es:di = vram ---
  mov  di, [vram_off]
  add  di, bp
  mov  es, [vram_seg]
  mov  al, byte ptr [col]
  mov  es:[di], al

dl_skip_put:
  ; --- break? ---
  mov  ax, [x0]
  cmp  ax, [x1]
  jne  dl_step
  mov  ax, [y0]
  cmp  ax, [y1]
  je   dl_end

dl_step:
  mov  ax, [dl_err]
  shl  ax, 1                ; ax = e2

  mov  bx, [dl_dy]
  neg  bx
  cmp  ax, bx
  jle  dl_check_y

  mov  bx, [dl_dy]
  sub  [dl_err], bx
  mov  bx, [dl_sx]
  add  [x0], bx
```

```
dl_check_y:
  cmp  ax, [dl_dx]
  jge  dl_loop

  mov  bx, [dl_dx]
  add  [dl_err], bx
  mov  bx, [dl_sy]
  add  [y0], bx

  jmp  dl_loop

dl_end:
  pop  di
  pop  si
  pop  bp
  pop  dx
  pop  cx
  pop  bx
  ret
 draw_line endp




 update_body proc near
  ; body.vel.y += body.g
  mov  ax, [body + BODY_G]
  add  [body + BODY_VEL + PT_Y], ax
  ; body.pos.x += body.vel.x
  mov  ax, [body + BODY_VEL + PT_X]
  add  [body + BODY_POS + PT_X], ax
  ; body.pos.y += body.vel.y
  mov  ax, [body + BODY_VEL + PT_Y]
  add  [body + BODY_POS + PT_Y], ax
  ; body.tri.angle += body.tri.dangle
  mov  ax, [body + BODY_TRI + TRI_DANGLE]
  add  [body + BODY_TRI + TRI_ANGLE], ax
  ;int r_factor = (body.tri.side * 148) >> 8;
  mov  ax, [body + BODY_TRI + TRI_SIDE]
  mov  bx, 148
  imul bx
  mov  cl, 8
  sar  ax, cl
```

```
  mov  [r_factor], ax
  ;int angle_idx = body.tri.angle & 255;
  mov  ax, [body+ BODY_TRI + TRI_ANGLE]
  and  ax, 255
  mov  [angle_idx], ax
  xor  si, si                              ; si = i = 0

point_loop:
  ; local_idx = (angle_idx + 64 + i * 85) & 255
  mov  ax, si                              ; ax = i
  mov  bx, 85
  imul bx                                 ; ax = i * 85
  add  ax, [angle_idx]                     ; ax += angle_idx
  add  ax, 64                              ; ax += 64
  and  ax, 255                             ; ax &= 255
  mov  [local_idx], ax                     ; сохранить

  ; wx = pos.x + ((r_factor * sin_tab[(local_idx+64)&255]) >> 8)
  add  ax, 64                              ; ax = local_idx + 64
  and  ax, 255
  shl  ax, 1                  ; <-- добавить: ax *= 2 (слово = 2 байта)
  mov  bx, offset sin_tab
  add  bx, ax
  mov  bx, [bx]                       ; bx = sin_tab[...]
  mov  ax, [r_factor]
  imul bx                                 ; dx:ax = r_factor * sin_tab
  mov  cl, 8
  sar  ax, cl                              ; ax >>= 8
  add  ax, [body + BODY_POS + PT_X]        ; ax += pos.x
  mov  di, ax                              ; di = wx

  ; wy = pos.y + ((r_factor * sin_tab[local_idx]) >> 8)
  mov  ax, [local_idx]
  shl  ax, 1                  ; <-- добавить: ax *= 2 (слово = 2 байта)
  mov  bx, offset sin_tab
  add  bx, ax
  mov  bx, [bx]                       ; bx = sin_tab[local_idx]
  mov  ax, [r_factor]
  imul bx
  mov  cl, 8
  sar  ax, cl
  add  ax, [body + BODY_POS + PT_Y]        ; ax += pos.y
  mov  bp, ax                              ; bp = wy
```

```
  ; points[i].x = wx, points[i].y = wy
  mov  ax, si                              ; ax = i
  shl  ax, 2                               ; ax = i * 4
  mov  bx, offset body + BODY_TRI + TRI_POINTS
  add  bx, ax                              ; bx = &points[i]
  mov  [bx + PT_X], di                     ; points[i].x = wx
  mov  [bx + PT_Y], bp                     ; points[i].y = wy

  ;  i++
  inc  si
  cmp  si, 3
  jl   point_loop
  call resolve_collision
  ret
 update_body endp



 resolve_collision proc near
  push cx
  push dx
  push bx
  ; int minX = w[0].x, maxX = w[0].x;
  ; int minY = w[0].y, maxY = w[0].y;
  mov  bx, offset body + BODY_TRI + TRI_POINTS
  mov  ax, [bx + PT_X]
  mov  word ptr [minX], ax
  mov  word ptr [maxX], ax
  mov  ax, [bx + PT_Y]
  mov  word ptr [minY], ax
  mov  word ptr [maxY], ax

  add  bx, PT_SIZE
  mov  cx, 2 ;первая точка уже лежит
rc_loop:
  ; if( w[i].x < minX )
  ; minX = w[i].x;
  mov  ax, [bx + PT_X]
  cmp  ax, [minX]
  jge  rc_max_x
  mov  word ptr [minX], ax
rc_max_x:
```

```
  ; if( w[i].x > maxX )
  ; maxX = w[i].x;
  mov  ax, [bx + PT_X]
  cmp  ax, [maxX]
  jle  rc_min_y
  mov  word ptr [maxX], ax
rc_min_y:
  ; if( w[i].y < minY )
  ;  minY = w[i].y;
  mov  ax, [bx + PT_Y]
  cmp  ax, [minY]
  jge  rc_max_y
  mov  word ptr [minY], ax
rc_max_y:
  ; if( w[i].y > maxY )
  ;  maxY = w[i].y;
  mov  ax, [bx + PT_Y]
  cmp  ax, [maxY]
  jle  rc_next
  mov  word ptr [maxY], ax
rc_next:
  add  bx, PT_SIZE
  loop   rc_loop


  mov  bx, offset body
  ; if( minX < LEFT_LIMIT )
  ; {
  ;  body.pos.x += ( LEFT_LIMIT - minX ); /* выталкиваем внутрь */
  ;  body.vel.x = -body.vel.x;            /* отражение */
  ; }
  cmp  word ptr [minX], 0
  jge  rc_right

  mov  ax, [minX]
  sub  word ptr [bx + BODY_POS + PT_X], ax

  mov  ax, [bx + BODY_VEL + PT_X]
  neg  ax
  mov  word ptr [bx + BODY_VEL + PT_X], ax
rc_right:
  ; if( maxX > RIGHT_LIMIT )
  ; {
```

```
  ;  body.pos.x -= ( maxX - RIGHT_LIMIT );
  ;  body.vel.x = -body.vel.x;
  ; }
  mov  ax, [maxX]
  cmp  ax, [viewW]
  jle  rc_top

  sub  ax, [viewW]
  sub  word ptr [bx + BODY_POS + PT_X], ax

  mov  ax, [bx + BODY_VEL + PT_X]
  neg  ax
  mov  word ptr [bx + BODY_VEL + PT_X], ax
rc_top:
  ;  if( minY < TOP_LIMIT )
  ; {
  ;  body.pos.y += ( TOP_LIMIT - minY );
  ;  body.vel.y = 0;
  ; }
  cmp  word ptr [minY], 0
  jge  rc_bottom

  mov  ax, [minY]
  sub  word ptr [bx + BODY_POS + PT_Y], ax

  mov  word ptr [bx + BODY_VEL + PT_Y], 0

rc_bottom:
  ;  if( maxY > BOTTOM_LIMIT )
  ; {
  ;  body.pos.y -= ( maxY - BOTTOM_LIMIT );
  ;  body.vel.y = -body.vel.y; /* отражение */
  ;  body.vel.y -= body.impulse;
  ; }
  mov  ax, [maxY]
  cmp  ax, [viewH]
  jle  rc_end

  sub  ax, [viewH]
  sub  word ptr [bx + BODY_POS + PT_Y], ax

  mov  ax, [bx + BODY_VEL + PT_Y]
  neg  ax
```

```
  mov  word ptr [bx + BODY_VEL + PT_Y], ax

  mov  ax, [bx + BODY_IMPULSE]
  sub  word ptr  [bx + BODY_VEL + PT_Y], ax
rc_end:
  pop  bx
  pop  dx
  pop  cx
  ret
 resolve_collision endp

_TEXT ENDS
; ============================================================
; КОНЕЦ ФАЙЛА
; ============================================================

 END main
```